

VM 4 - Remediation Tracker and Evidence Checklist

Supporting document for the penetration testing project

Target VM: CentOS 7 Docker host | Applications: Portainer and OctoberCMS

Purpose

This document supports the final VM 4 penetration testing report. It tracks confirmed findings, evidence commands, remediation actions, and verification steps in a clean project-ready format.

Executive Remediation Summary

ID	Finding	Severity	Status
1	Exposed Portainer Docker Management Interface	Critical	Remediated
2	Docker Socket Mounted into Portainer	High / Critical	Mitigated
3	Outdated Portainer Image	High	Partially remediated
4	Exposed Weak MySQL Root Credentials	Critical	Remediated
5	Sensitive Information Disclosure in Logs	High	Mitigated
6	Container Running as Root User	Medium / High	Remediated
7	Outdated OctoberCMS Image / Legacy Components	High	Requires rebuild

Key message: the most serious risk was the exposed Portainer Docker management interface, supported by exposed database credentials in Docker metadata. These findings create a realistic path from management access to sensitive data exposure.

Detailed Finding Tracker

1. Exposed Portainer Docker Management Interface

Severity: Critical

Evidence:

- Portainer was exposed on ports 8000 and 9000.

- Portainer login page was reachable from the browser.
- Docker showed published ports bound to 0.0.0.0.

Commands used:

```
docker ps -a
nmap -sV -sC -p 8000,9000 192.168.56.101
Test-NetConnection 192.168.56.101 -Port 9000
```

Remediation: External access was blocked and Portainer was recreated with ports bound to 127.0.0.1 only.

Remediation / verification commands:

```
iptables -I DOCKER-USER -i enp0s17 -p tcp --dport 9000 -j DROP
iptables -I DOCKER-USER -i enp0s17 -p tcp --dport 8000 -j DROP
docker run -d -p 127.0.0.1:8000:8000 -p 127.0.0.1:9000:9000 --name portainer --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer
```

2. Docker Socket Mounted into Portainer

Severity: High / Critical

Evidence:

- Portainer had /var/run/docker.sock mounted.
- This gives Portainer Docker daemon management capability.

Commands used:

```
docker inspect portainer | grep -i "/var/run/docker.sock"
docker inspect portainer | grep -A 10 '"Binds"'
```

Remediation: Portainer access was restricted to localhost, and secure administration should use SSH tunneling or a VPN.

Remediation / verification commands:

```
ssh -L 9000:127.0.0.1:9000 root@192.168.56.101
```

3. Outdated Portainer Image

Severity: High

Evidence:

- The Portainer image was portainer/portainer:latest.
- The image was created in 2019 and shown as around 6 years old.

Commands used:

```
docker images | grep -i portainer
docker image inspect portainer/portainer | grep -Ei "Created|Version|Maintainer|RepoTags"
```

Remediation: Upgrade to portainer/portainer-ce:latest was attempted. The pull failed due to CA/DNS trust issues, so local-only binding remains as the temporary control.

Remediation / verification commands:

```
docker stop portainer
docker rm portainer
docker pull portainer/portainer-ce:latest
```

4. Exposed Weak MySQL Root Credentials

Severity: Critical

Evidence:

- Docker metadata exposed DB_USERNAME=root and DB_PASSWORD=root.
- The database name octobercms and host 127.0.0.1 were also visible.

Commands used:

```
docker inspect vulnerable_october_restarting | grep -Ei "DB_USERNAME|DB_PASSWORD|DB_DATABASE|DB_HOST"
docker inspect vulnerable_october_port80 | grep -Ei "DB_USERNAME|DB_PASSWORD|DB_DATABASE|DB_HOST"
```

Remediation: The vulnerable containers were stopped and OctoberCMS was recreated using a dedicated database user and stronger password.

Remediation / verification commands:

```
docker run -d --name october_secure -p 127.0.0.1:80:80 -e DB_TYPE=mysql -e DB_HOST=127.0.0.1 -e DB_DATABASE=octobercms -e
DB_USERNAME=october_user -e DB_PASSWORD='StrongOctoberDB@2026' octobercms:1.0.412
```

5. Sensitive Information Disclosure in OctoberCMS Logs

Severity: High

Evidence:

- Logs exposed backend routes, stack traces, MySQL errors, and sensitive user update values.
- The logs included authentication activity and internal application errors.

Commands used:

```
docker logs vulnerable_october_port80 2>&1 | tail -50
docker logs vulnerable_october_port80 2>&1 | grep -Ei "password|user|login|signin|error|traceback|exception|mysql|database|key|token"
```

Remediation: Vulnerable containers were stopped/renamed for evidence, and the active service should disable debug output and restrict Docker log access.

Remediation / verification commands:

```
docker stop vulnerable_october_port80 vulnerable_october_restarting
grep -Rni "debug" /var/www /var/www/html /app 2>/dev/null
```

6. Container Running as Root User

Severity: Medium / High

Evidence:

- Docker inspect showed an empty User field.
- docker exec october_secure id returned uid=0(root) before remediation.

Commands used:

```
docker inspect october_secure | grep -Ei '"User"|Privileged|UsernsMode'
docker exec october_secure id
```

Remediation: The container was recreated with --user 1000:1000 and verified as uid=1000.

Remediation / verification commands:

```
docker stop october_secure
docker rm october_secure
docker run -d --name october_secure --user 1000:1000 -p 127.0.0.1:80:80 -e DB_TYPE=mysql -e DB_HOST=127.0.0.1 -e DB_DATABASE=octobercms -e
DB_USERNAME=october_user -e DB_PASSWORD='StrongOctoberDB@2026' octobercms:1.0.412
docker exec october_secure id
```

7. Outdated OctoberCMS Image / Legacy Components

Severity: High

Evidence:

- OctoberCMS image octobercms:1.0.412 was present.
- Legacy runtime components were identified in logs and container checks.

Commands used:

```
docker images | grep -i october
docker exec october_secure apache2 -v
docker exec october_secure php -v
docker exec october_secure mysqld --version
```

Remediation: Rebuild or replace the image with supported OctoberCMS, PHP, Apache, and MySQL versions. Keep local-only binding until the rebuild is tested.

Remediation / verification commands:

```
docker run -d --name october_updated -p 127.0.0.1:80:80 <updated-octobercms-image>
```

Recommended Submission Package

- Final Penetration Testing Report PDF
- This remediation tracker and evidence checklist
- Screenshots of before-and-after verification commands
- Optional executive summary for non-technical review

Final Notes

The strongest findings are the exposed Portainer Docker management interface and the exposed MySQL root credentials in Docker metadata. Together, they show a serious attack path from Docker management access to sensitive credential retrieval and possible database compromise.